

FAST · arXiv 2501.09747 · Physical Intelligence

Archaeologist

11+3 · seat 3

Naming hygiene π_0 / π_0 -FAST / $\pi_{0.5}$

FAST $\rightarrow$ π_0 -FAST $\rightarrow$ $\pi_{0.5}$ $\rightarrow$ KI $\rightarrow$ RTC $\rightarrow$ BEHAVIOR

Oct 19 paper club · VLA Architectures

MYTH BUST: FAST $\neq$ FM · $\pi_{0.5}$ = FAST pretrain $\rightarrow$ FM deploy

$\pi_0 \approx$ FM · π_0 -FAST $\approx$ discrete AR · $\pi_{0.5} \approx$ hybrid successor

CRITICAL MYTH BUST · say this first

FAST does NOT replace flow matching at deploy

Label	Meaning
FAST	Discrete DCT→BPE action tokenizer for AR VLAs
FM	Continuous flow-matching (π_0 / $\pi_{0.5}$ deploy)
π_0 -FAST	Discrete AR variant using this tokenizer
$\pi_{0.5}$	FAST for pretrain · FM for post-train/deploy

Naming: $\pi_0 \approx \text{FM}$ · π_0 -FAST $\approx$ discrete AR · $\pi_{0.5} \approx$ hybrid successor

Naming hygiene tree · Archaeologist / club spine

Never collapse these three labels

π_0

≈ continuous FM
PaliGemma + action expert
~100 ms / chunk deploy

π_0 -FAST

≈ discrete AR sibling
Same backbone + FAST tokens
~750 ms / chunk · 5× train

$\pi_{0.5}$

≈ hybrid successor
FAST pretrain → FM post
FM RETAINED at deploy

Forward lineage (CLEAR)

FAST

π_0 -FAST

$\pi_{0.5}$
(FAST→FM)

KI

RTC

BEHAVIOR

Intellectual ancestry · Archaeologist

DCT / JPEG

Ahmed 1974 · Wallace JPEG
signal compression

BPE

Gage · Sennrich et al.
LLM subword compression

Naïve bins

RT-1 / RT-2 / OpenVLA
per-dim per-timestep

Action chunks

ACT · Diffusion Policy · π_0
~1 s open-loop prefixes

Continuous FM

π_0 2410.24164
motivated the race

Learned discrete

VQ-VAE / FSQ
paper: DCT simpler + better

FAST pipeline · DCT → BPE

Quantile norm → DCT → scale γ → round → low-freq-first flatten → BPE $|V|=1024$

① Norm

1-99% quantile
→ [-1,1]

② DCT

per-dim
over H

③ Quant

$\text{round}(\gamma \cdot C)$
 $\gamma=10$ default

④ Flatten

low-freq
across dims first

⑤ BPE

vocab 1024
~30 tok/arm

Why it works

- ▶ Naïve bins at high Hz → copy-last-token
- ▶ DCT packs shape into low-freq coeffs
- ▶ Sparse zeros → BPE squashes stream
- ▶ Invertible decode → continuous actions
- ▶ Overwrite least-used VLM vocab slots

Defaults / HF

Chunk $\approx$ 1 second

$\gamma = 10$ · $|V| = 1024$

Pad dims to 32 (FAST+)

```
AutoProcessor.from_pretrained(  
    "physical-intelligence/fast",  
    trust_remote_code=True)
```

When FAST helps vs when continuous FM wins

FAST helps

- ▶ High-Hz next-token prediction
- ▶ Train compute / compression
- ▶ Denser chunks without token blowup
- ▶ Language-following AR VLAs
- ▶ Cheap large-scale discrete pretrain
- ▶ $\pi_{0.5}$ discrete pretrain phase

Continuous FM wins

- ▶ Inference latency / smooth dynamics
- ▶ Closed-loop control (fewer ms/chunk)
- ▶ Real-time household deploy
- ▶ ~ 100 ms vs ~ 750 ms on 4090
- ▶ π_0 / $\pi_{0.5}$ post-train & serve
- ▶ Contact-rich multimodal density

$\pi_{0.5}$ recipe: FAST for efficient pretrain NTP → serve with continuous FM

Club one-liner

FAST = discrete DCT→BPE tokenizer for AR VLAs.

π_0 -FAST matches FM π_0 on dexterity at $\sim 5\times$ less train compute,
but infer ~ 750 ms vs FM ~ 100 ms.

$\pi_{0.5}$ keeps FAST for pretrain and FM for deploy.

Do NOT delete FM because FAST exists.

Fig.2 pipeline + score vs Hz

future actions, i.e., action “chunks”, as is common for high-frequency control. Highly correlated action tokens *diminish* the effectiveness of the next token prediction objective used in autoregressive VLAs. Intuitively, in such cases low token prediction loss can often be achieved with mappings as trivial as simply copying the most recent action token, leaving models in poor local optima.

In this work, we propose a new tokenization strategy from first principles. Our key insight is that robot action signals need to be *compressed* before training, to reduce correlation between consecutive tokens. We take inspiration from compression-based tokenization strategies, such as the byte-pair encoding method commonly used by language models [27, 57]. However, since robotic actions are continuous, the corresponding compression strategy should be chosen accordingly. We therefore base our method off of the discrete cosine transform (DCT) encoding, which is widely used for compressing continuous signals such as images (e.g., JPEG compression). We find that the resulting tokenization approach, **F**requency-space **A**ction **S**equences **T**okenization (**FAST**), enables us to train autoregressive VLA policies via simple next token prediction (see Figure 2, left) for highly dexterous and high-frequency tasks where standard discretization methods fail entirely. Additionally, FAST for the first time enables efficient VLA training on the recently introduced DROID dataset [38], a large-scale multitask “in-the-wild” robot manipulation dataset. The resulting policy is the first language-conditioned generalist manipulation policy that can be successfully evaluated *zero-shot* in unseen environments, simply by prompting it in natural language.

Based on FAST, we develop FAST+, a **universal robot action tokenizer**, trained on 1M real robot action trajectories that cover a large diversity of robot embodiments, action spaces and control frequencies. We demonstrate that the FAST+ tokenizer effectively tokenizes a wide range of robot action sequences, from single-arm to bi-manual and mobile robots, and is a good off-the-shelf tokenizer for training autoregressive VLA models. When integrated with the π_0 VLA, FAST-based autoregressive VLAs scale to training on 10k hours of robot data and achieve performance comparable to diffusion-based VLAs across a variety of tasks, while reducing training time by up to 5x (see Figure 1).

II. RELATED WORK

Tokenization for language, text, and audio. Tokenization is a key component of training pipelines for modern transformer-based autoregressive sequence models, and the choice of tokenization approach can have significant impact on model training and downstream performance [55]. While there are multiple works exploring the training of “tokenization-free” language models [28, 53] that directly operate on bit streams, most language models today rely on a text tokenization stage prior to training. A common approach is byte pair encoding [27, 55], which compresses input text by merging frequently occurring token sequences into new tokens. For images, *learned* compression schemes present an effective

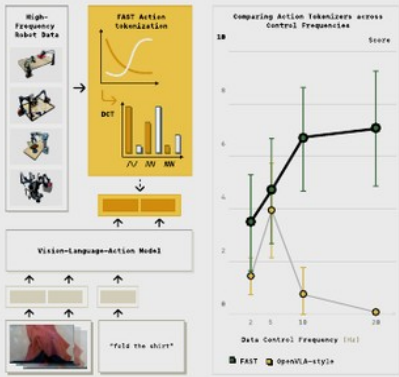


Fig. 2: **Left:** FAST tokenization enables training of autoregressive Transformers for dexterous robot control via simple next token prediction. **Right:** FAST outperforms popular binning tokenization schemes, e.g., used in OpenVLA [39], particularly for high-frequency robot data.

approach: input images can be represented as “soft tokens” produced by a pre-trained vision encoder [44], and full autoregressive image input-output can be achieved with a vector-quantizing autoencoder [22, 59]. Similar approaches can be extended to the video domain [66]. In audio generation and speech synthesis, which share the time-series structure of action prediction, state-of-the-art models typically encode time-series audio data using either frequency-domain spectrogram images [29] or using learned vector quantizers [68].

Vision-language-action models. Recently, multiple works have developed *generalist* robot policies [9, 51, 6, 10, 20, 39, 62, 11] that are trained on increasingly large robot learning datasets [52, 38, 60, 24, 47, 35]. One promising approach for training generalist policies are vision-language-action models (VLAs: [10, 17, 39, 67, 7, 63, 73, 71, 13, 11]). VLAs fine-tune vision-language models, that are pre-trained on internet-scale image and text data, for robot control. This has multiple benefits: using large vision-language model backbones, with billions of parameters, provides policies with the necessary expressivity for fitting large robot datasets. Reusing weights pre-trained on internet-scale datasets also improves the ability of VLAs to follow diverse language commands and generalize, e.g., to new objects and scene backgrounds [10, 39, 67, 63, 36]. Most VLA models today are confined to rather simple, low-frequency control tasks, particularly models that use the most common autoregressive VLA design [10, 39]. We show that this is a direct consequence of the *action tokenization* schemes employed by these models, which make training on dexterous tasks challenging. We introduce a new action tokenization

Fig.4 DCT→BPE pipeline + Alg.1

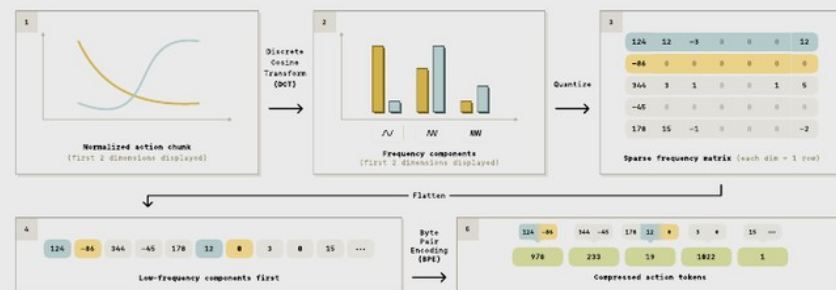


Fig. 4: Overview of the FAST action tokenization pipeline. Given a normalized chunk of actions, we apply discrete cosine transform (DCT) to convert the signal to the frequency domain. We then quantize the DCT coefficients and use byte-pair encoding (BPE) to compress the flattened sequence of per-dimension DCT coefficients into the final action token sequence. See Section V-B for a detailed description.

matrix into a sequence of dense tokens. We flatten the matrix into a 1-dimensional vector of integers, interleaving action dimensions by including all low-frequency components first, and train a byte pair encoding (BPE) tokenizer [27] to losslessly compress it into dense action tokens. The BPE step “squashes” the zero-valued components and merges frequently-occurring coefficient combinations across action dimensions. We choose BPE to compress the DCT matrix, since many efficient implementations exist and it can produce a fixed-size output vocabulary that can be easily integrated into the existing vocabulary of vision-language models for VLA training. Other lossless compression algorithms like Huffman coding [33] or Lempel-Ziv methods [75] (the algorithms underlying the gzip compression approach) could be used instead, but we leave this investigation for future work.

Note that the *order* of flattening the $|A| \times H$ DCT coefficient matrix prior to BPE encoding can have significant impact on policy training. There are two options: column-first flattening, i.e., concatenate the lowest-frequency components for each dimension first, or row-first flattening, i.e., concatenating all frequency components for a single action dimension first. We choose the former, since we find that predicting the *low-frequency* components, that characterize the overall shape of the output sequence, first during autoregressive prediction leads to more stable policy rollouts.

All operations in our tokenization pipeline are easily invertible, allowing fast decoding of predicted actions. The tokenizer has only two hyperparameters: the scale applied to the DCT coefficients before rounding, and the vocabulary size of the BPE compression step. We find that both parameters are not very sensitive, and we use the same values across all our single-dataset tokenization experiments (rounding scale 10, BPE vocabulary size 1024). This is in contrast to end-to-end *learned* compression modules that rely on vector quantiza-

Algorithm 1 FAST Tokenizer

Require: scale γ , (for inference) BPE dictionary Φ
procedure FASTTOKENIZER($a_{1:H}$)
 $C^1 \leftarrow \text{DCT}(a_{1:H})$ $\triangleright$ Compute DCT coefficients
 $C^j \leftarrow \text{round}(\gamma \cdot C^j)$ $\triangleright$ Quantize coefficients
 $[T_k] \leftarrow [C^1, C^2, \dots, C^1, \dots, C^H]$ $\triangleright$ Flatten tokens
BPE Training:
 $\phi \leftarrow \text{TrainBPE}(\mathcal{D} := \{[T_k]\})$
Tokenization:
 $[T_1, \dots, T_k] \leftarrow \text{BPE}([T_1, \dots, T_k], \phi)$
return action_tokens

tion [59]. Such networks are often tedious to train, and require careful dataset-specific hyperparameter selection to achieve good reconstruction [66, 48]. Our experiments show that our DCT-based tokenization approach trains higher-performing policies than VQ-based approaches, while being significantly simpler and easier to tune.

We empirically demonstrate the benefits of our DCT-based tokenization in the toy example from Section IV. Figure 3 shows that training the autoregressive model on DCT-compressed target tokens achieves constantly low prediction error across a wide range of sampling frequencies. We provide a concise summary of our tokenization approach in Algorithm 1 and test the effectiveness of FAST tokenization on robot control problems in Section VI.

C. A Universal Robot Action Tokenizer

The only *learned* component of our tokenizer is the vocabulary of the BPE encoder, which needs to be trained for each new dataset that the tokenizer is being applied to. While this learning process is fast (typically only a few minutes), it adds additional friction to using FAST tokenization. Thus,

arXiv:2501.09747v1 [cs.RO] 16 Jan 2025

FAST: Efficient Action Tokenization for Vision-Language-Action Models

Karl Pertsch^{*,1,2,3}, Kyle Stachowicz^{*,2},
Brian Ichter¹, Danny Driess¹, Suraj Nair¹, Quan Vuong¹, Oier Mees², Chelsea Finn^{1,3}, Sergey Levine^{1,2}
¹Physical Intelligence, ²UC Berkeley, ³Stanford
<https://pi.website/research/fast>

Abstract—Autoregressive sequence models, such as Transformer-based vision-language action (VLA) policies, can be tremendously effective for capturing complex and generalizable robotic behaviors. However, such models require us to choose a tokenization of our continuous action signals, which determines how the discrete symbols predicted by the model map to continuous robot actions. We find that current approaches for robot action tokenization, based on simple per-dimension, per-timestep binning schemes, typically perform poorly when learning dexterous skills from high-frequency robot data. To address this challenge, we propose a new compression-based tokenization scheme for robot actions, based on the discrete cosine transform. Our tokenization approach, Frequency-space Action Sequence Tokenization (FAST), enables us to train autoregressive VLAs for highly dexterous and high-frequency tasks where standard discretization methods fail completely. Based on FAST, we release FAST+, a *universal* robot action tokenizer, trained on 1M real robot action trajectories. It can be used as a black-box tokenizer for a wide range of robot action sequences, with diverse action spaces and control frequencies. Finally, we show that, when combined with the π_0 VLA, our method can scale to training on 10k hours of robot data and match the performance of diffusion VLAs, while reducing training time by up to 5x.

I. INTRODUCTION

Large, high-capacity Transformer models can be tremendously effective for capturing complex and generalizable robotic behaviors both from scratch [8, 69, 51, 6, 20, 62] and using models pre-trained for next-token prediction on Internet-scale image-text corpora [10, 39, 63, 7, 65]. However, these models require choosing a tokenization of the continuous action signal, which determines how the discrete symbols predicted by the model map to continuous robot actions [64, 34, 41, 12]. It is widely known that a good choice of tokenization can be critical to the performance of sequence models [55, 57]. Prior robotic policies of this sort typically use naïve tokenization strategies based on a per-dimension, per-timestep binning scheme [9, 10, 39]. We find that such methods perform poorly when learning dexterous skills with high-frequency control (see Figure 2, right). We observe that correlations between time steps are a major challenge for naïve tokenization strategies when predicting sequences of

^{*}: Core contributors
Correspondence to: research@physicalintelligence.com

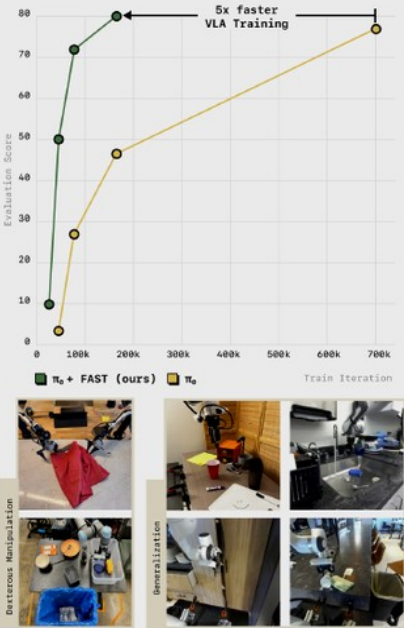


Fig. 1: We propose FAST, a simple yet effective approach for tokenization of robot action trajectories via time-series compression. FAST enables training of autoregressive VLAs that solve complex dexterous manipulation tasks and generalize broadly to new scenes. We use it to train π_0 -FAST, a generalist robot policy that matches the performance of the state-of-the-art π_0 diffusion VLA on dexterous and long-horizon manipulation tasks, while training 5x faster (top).

Hybrid for 2026 BEHAVIOR

FM(+RTC) deploy · FAST pretrain/aux · no Discord/Drive

1

Keep FM(+RTC)

Smooth multi-second household skills + latency at serve

2

Keep FAST

Cheap AR co-train, language, tokenizer compression research

3

AR decode speed

Speculative decode / quant / draft discrete VLAs compete dynamically

4

FAST+ default

few morphologies / sims entering challenge without training VQ

5

Aux loss pattern

B1K'25: FAST aux $w=0.05$ on $\pi_{0.5}$ FM primary — hybrid wins

Skip Discord/Drive — cite paper + HF + PI blog + $\pi_0/\pi_{0.5}$ packs

Fig.7-8 DROID ZS + FAST+ OOD



Fig. 6: Comparison of policy performance using different tokenization approaches. We find that tokenization approaches that compress action targets (FAST, FSQ) lead to substantially more efficient training than the naïve binning tokenization used in prior VLAs. Overall, we find that FAST leads to more effective policy training than FSQ, particularly on dexterous real-robot tasks. Our universal tokenizer, FAST+, matches the performance of dataset-specific tokenizers. We report mean and 95% CI.

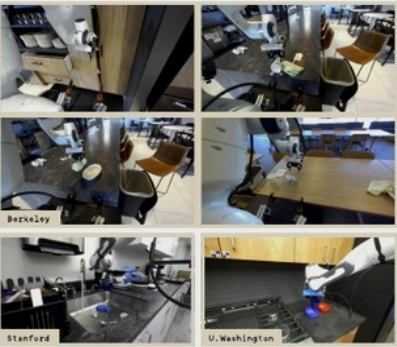


Fig. 7: Evaluation environments of FAST policy trained on DROID [38]. We find that the same policy checkpoint generalizes robustly, and performs various simple table-top tasks *zero-shot* across three university campuses.

baseline, we find that FAST is as good or at times better, particularly on the dexterous, high-frequency tasks, despite being much simpler and requiring no separate neural network training.

Notably, FAST tokenization enables the first successful training of a strong generalist policy on the DROID dataset [38], which can be evaluated *zero-shot* in unseen environments, without fine-tuning, by simply prompting it in natural language. All prior works, including the original DROID paper [38] and OpenVLA [39], did not show zero-shot results and focused entirely on co-training or fine-tuning evaluations instead. We demonstrate the generality of our DROID policy by testing it on various table-top manipulation tasks

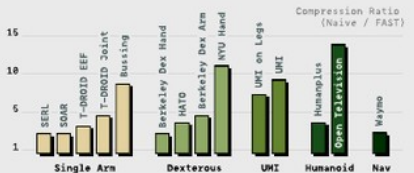


Fig. 8: Universal tokenizer. We test the compression rate achieved by our FAST+ tokenizer vs. naïve tokenization across diverse robot datasets, *unseen* during tokenizer training. We find that FAST is effective across a wide range of robot morphologies, action spaces and control frequencies.

in environments across three university campuses (Figure 7). Out of the box, the policy can competently perform simple manipulation tasks, like picking and placing objects, opening and closing cupboards and turning on faucets, across a wide range of scenes and camera viewpoints. Even unsuccessful trials show sensible behavior, like approaching the handles of microwave and dish washer doors, even if ultimately failing to open them. We show success and failure videos on our website. While far from perfect, the level of generality and robustness of this policy substantially exceeds that of prior DROID policies.

C. Universal Action Tokenizer

In this section, we evaluate the performance of our *universal* action tokenizer, FAST+, which we trained on 1M real robot action sequences (see Section V-C). To test the *generality* of the tokenizer, we assemble a diverse set of small testing datasets. This set spans a wide range of robot morphologies, action spaces, and control frequencies (see Figure 8, with a full list of datasets in Table III). Note that none of these datasets is part of the tokenizer training set. They thus test a scenario in

Archaeologist · close

Keep labels distinct: π_0 / π_0 -FAST / $\pi_{0.5}$.

Lineage: FAST $\rightarrow$ π_0 -FAST $\rightarrow$ $\pi_{0.5}$ (FAST pre $\rightarrow$ FM) $\rightarrow$ KI $\rightarrow$ RTC $\rightarrow$ BEHAVIOR

Q&A · Oct 19 · FAST / 2501.09747

Naming hygiene tree · Archaeologist / club spine

Never collapse these three labels

π_0

≈ continuous FM
PaliGemma + action expert
~100 ms / chunk deploy

π_0 -FAST

≈ discrete AR sibling
Same backbone + FAST tokens
~750 ms / chunk · 5× train

$\pi_{0.5}$

≈ hybrid successor
FAST pretrain → FM post
FM RETAINED at deploy

Forward lineage (CLEAR)

FAST

π_0 -FAST

$\pi_{0.5}$
(FAST→FM)

KI

RTC

BEHAVIOR